



Department of Electronics & Communications

25SC2107E – Machine Learning

Practical Session

Gradient Descent from Scratch — CGPA Regression

Dataset: PlacementPredict | Course Outcome: CO2 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment 3: Gradient Descent from Scratch — CGPA Regression

Dataset: PlacementPredict

Course Outcome: CO2

1.1 Aim

To implement batch and mini-batch gradient descent in NumPy for the prediction of student CGPA, and to verify the resulting parameters against the closed-form solution of `LinearRegression`.

1.2 Learning Objectives

After completing this experiment, students should be able to:

- Write the gradient descent update rule in NumPy without using scikit-learn.
- Plot a loss curve and name the fault behind each failure shape.
- Distinguish an *update*, an *epoch* and the *batch size*.
- Explain the difference between batch and mini-batch gradient descent.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib scikit-learn
```

1.4 Dataset

PlacementPredict, supplied as `placement_predict_50k.csv` in this folder. Keep the file beside your notebook.

- 50,000 rows, one per student. Nothing is missing, so there is no cleaning to do.
- **Answer column:** CGPA, a number, so this is a **regression** problem.
- **Input columns:** Internships, Projects, AptitudeScore, SoftSkillsRating, Backlogs.

1.5 Procedure

1. Create a notebook `tut_lab_LR_GD.ipynb` in this folder.
2. Load the data and inspect its structure.
3. Choose the input columns and the answer column, then split into training and validation sets.
4. Scale the inputs. Gradient descent will not work without this step.
5. Fit `LinearRegression` to obtain the reference answer.
6. Write batch gradient descent in NumPy and plot its loss curve.
7. Compare the hand-written coefficients with scikit-learn's.
8. Write mini-batch gradient descent and plot both curves together.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import mean_squared_error, r2_score
```

- numpy does the maths. Here it does the *training* as well, which is new.
- `LinearRegression` is imported only as a *reference answer* to check your own work against.

Cell 2 — Load the data and check the structure

```
1 placement = pd.read_csv("placement_predict_50k.csv")
2 print(placement.shape)
3 placement.info()
4 print(placement.describe())
```

- `shape` prints (50000, 12). Every `info()` count reads 50,000, so nothing is missing.
- Confirm CGPA runs from about 5.0 to 10.0. Error is only meaningful against the scale of the answer: being wrong by 0.5 of a grade point is a lot.

Cell 3 — Choose the columns and split

```
1 features = ["Internships", "Projects", "AptitudeScore",
2            "SoftSkillsRating", "Backlogs"]
3
4 X = placement[features]
5 y = placement["CGPA"]
6
7 X_train, X_val, y_train, y_val = train_test_split(
8     X, y, test_size=0.2, random_state=42)
9 print(X_train.shape, X_val.shape)
```

- CGPA is the answer, so it must never appear in X.
- `test_size=0.2` gives **40,000** rows to learn from and **10,000** to test on.

Cell 4 — Scale the inputs

```
1 scaler = StandardScaler()
2 X_train_s = scaler.fit_transform(X_train)
3 X_val_s = scaler.transform(X_val)
4 print(X_train_s.mean().round(3), X_train_s.std().round(3))
```

- Fit on train, transform on both. The printout confirms it: mean 0, spread 1.
- **Without this the next cell fails.** `AptitudeScore` runs to 100 while `SoftSkillsRating` runs to 5, and gradient descent uses **one step size for every input at once**. A step safe for one is useless for the other.

Cell 5 — The reference answer

```
1 lr = LinearRegression()
2 lr.fit(X_train_s, y_train)
3
4 sk_pred = lr.predict(X_val_s)
5 sk_rmse = np.sqrt(mean_squared_error(y_val, sk_pred))
6 print("sklearn RMSE:", round(sk_rmse, 4))
7 print("sklearn R2  :", round(r2_score(y_val, sk_pred), 4))
```

- RMSE is about 0.51, so the model is typically wrong by half a grade point. R^2 is about 0.74.
- Scikit-learn does *not* use gradient descent here. It solves the equations directly in one step. The two methods reach the same place by different roads.

Note**The two scores**

RMSE = $\sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$ — *by how much are we typically wrong?* Square the errors so signs cannot cancel, average, then square-root back to the original units. So it is measured **in CGPA points**. Smaller is better; 0 is perfect.

$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$ — *how much better than guessing the average?* The denominator is the error you would make by ignoring every input and always predicting the mean CGPA. That is the regression **baseline**, exactly as 62% was the baseline in Experiment 1.

R^2	Meaning
1.0	every prediction exactly right
0.74	our model
0.0	no better than always guessing the average
negative	worse than guessing the average

R^2 has no units and travels between problems; RMSE has units and is the one to quote to a person.

Cell 6 — Batch gradient descent, written by hand

```

1 def batch_gd(X, y, epochs=1000, rate=0.1):
2     w = np.zeros(X.shape[1])
3     b = 0.0
4     y = np.asarray(y, dtype=float)
5     losses = []
6
7     for step in range(epochs):
8         error = X @ w + b - y
9         losses.append((error ** 2).mean())
10        w = w - rate * (X.T @ error) / len(y)
11        b = b - rate * error.mean()
12
13    return w, b, losses
14
15
16 w_batch, b_batch, loss_batch = batch_gd(X_train_s, y_train)
17 print("Final MSE:", round(loss_batch[-1], 4))

```

This is the entire training algorithm.

- **w** holds one weight per input; **b** is the starting height. Both begin at zero, meaning “predict nothing”.
- **X @ w + b** predicts all 40,000 students at once (@ is matrix multiplication).
- The two update lines move **w** and **b** a small distance downhill. One execution of those lines is **one update**.
- “Batch” means the whole training set is used in every step, so each update is accurate but costs a full pass over the data.

Note**Three words that get used loosely**

- **one update** — the parameters move once. Also called a *step* or an *iteration*.
- **one epoch** — one complete pass over the data, so every row has been used once.
- **batch size** — how many rows are used to compute a single update.

$$\text{updates in one epoch} = \frac{\text{number of rows}}{\text{batch size}}$$

Batch gradient descent uses the whole set, so one epoch is **one** update. The deliverable uses 32 rows, so one epoch is $40,000/32 = 1,250$ updates. **An epoch is not an update.**

Cell 7 — The loss curve

```
1 plt.plot(loss_batch)
2 plt.xlabel("epoch")
3 plt.ylabel("mean squared error")
4 plt.title("Batch Gradient Descent")
5 plt.show()
```

It should fall steeply, then flatten. Flattening means **converged**. The shape names the fault:

- **Still falling steeply at epoch 1000** → stopped too early, or **rate** too small.
- **Rises, wobbles, or becomes NaN** → **rate** too large.
- **Flat from the first epoch** → the inputs were not scaled.

Cell 8 — Did the hand-written version work?

```
1 gd_pred = X_val_s @ w_batch + b_batch
2 gd_rmse = np.sqrt(mean_squared_error(y_val, gd_pred))
3
4 print("sklearn RMSE      :", round(sk_rmse, 4))
5 print("batch GD RMSE     :", round(gd_rmse, 4))
6 print("largest coef gap  :", round(np.abs(w_batch - lr.coef_).max(), 6))
```

- The two RMSE values agree to four decimal places and the largest coefficient gap is essentially zero.
- **Ten lines of NumPy reproduce a library function exactly.** You have shown your code is *correct*, not that it is fast.
- **Backlogs** carries a negative weight and the other four are positive, which matches common sense.

Deliverable Function

```
1 def minibatch_gd(X, y, epochs=1000, batch_size=32, rate=0.01):
2     rng = np.random.default_rng(42)
3     w = np.zeros(X.shape[1])
4     b = 0.0
5     y = np.asarray(y, dtype=float)
6     n = len(y)
7     losses = []
8
9     for step in range(epochs):
10        order = rng.permutation(n)           # reshuffle each epoch
11        for start in range(0, n, batch_size):
12            idx = order[start:start + batch_size]
13            error = X[idx] @ w + b - y[idx]
14            w = w - rate * (X[idx].T @ error) / len(idx)
15            b = b - rate * error.mean()
16            losses.append(((X @ w + b - y) ** 2).mean())
17
18        return w, b, losses
19
20
21 w_mini, b_mini, loss_mini = minibatch_gd(X_train_s, y_train)
22
```

```

23 mini_rmse = np.sqrt(mean_squared_error(y_val, X_val_s @ w_mini +
    b_mini))
24 print("sklearn RMSE      :", round(sk_rmse, 4))
25 print("mini-batch RMSE :", round(mini_rmse, 4))
26
27 assert abs(mini_rmse - sk_rmse) < 0.3, "mini-batch RMSE is not within
    0.3 of sklearn"
28 print("Tolerance check PASSED")
29
30 plt.plot(loss_batch, label="batch")
31 plt.plot(loss_mini, label="mini-batch")
32 plt.xlabel("epoch")
33 plt.ylabel("mean squared error")
34 plt.legend()
35 plt.title("Batch vs Mini-batch")
36 plt.show()

```

The only change is *how much data is used per update*.

- Batch used all 40,000 rows per update: 1000 updates in total. Mini-batch uses 32, giving 1,250 updates *per epoch* and 1.25 million overall.
- `rng.permutation(n)` reshuffles every epoch so the batches differ. Without it the same 32 students would travel together on every pass.
- Each update costs 32 rows instead of 40,000. That matters when data will not fit in memory, which is why mini-batch is how large models are really trained. The price is a noisier path.
- The `assert` states a **tolerance**: mini-batch is random, so demanding an exact match would be wrong. Silence means success.

This cell takes ten to fifteen seconds. That is expected: it performs 1.25 million updates.

1.7 Expected Output

```

(50000, 12)
(40000, 5) (10000, 5)
sklearn RMSE: 0.5106
sklearn R2   : 0.7448
Final MSE: 0.2618
sklearn RMSE      : 0.5106
batch GD RMSE     : 0.5106
largest coef gap  : 0.0
mini-batch RMSE   : 0.5107
Tolerance check PASSED

```

1.8 Result

Batch gradient descent implemented in NumPy converged to a validation RMSE of 0.5106, with coefficients agreeing with `LinearRegression` to within 10^{-6} . The mini-batch variant, using 32 rows per update over 1000 epochs, attained an RMSE of 0.5107 and satisfied the prescribed tolerance of 0.3.

Note

Batch and mini-batch

	Batch	Mini-batch
Rows used per update	all 40,000	32
Updates per epoch	1	1,250
Path downhill	smooth	wobbly
Fits in memory?	only if data is small	always

Using *one* row per update is called **stochastic** gradient descent. Mini-batch sits between the two extremes, which is why it is the one used in practice.